



计算机网络实验报告

警示

1. 实验报告如有雷同，雷同各方当次实验成绩均以 0 分计。
2. 当次小组成员成绩只计学号、姓名登录在下表中的。
3. 在规定时间内未上交实验报告的，不得以其他方式补交，当次成绩按 0 分计。
4. 实验报告文件以 PDF 格式提交。

院系	计算机学院	班 级	计科 1 班	组长	陈安康
学号	22320021	22318001	22336070	22336064	
学生	陈安康	蔡可忻	冯乙山	杜雨彤	

编程实验

【实验内容】

(1) 参考教材 P98 例 3-1 内容，基于 P67 图 2-10 实验拓扑，完成 P103 实验教程实例 3-2 的实验（考虑局域网、互联网两种实验环境），回答实验提出的问题及实验思考。

(2) 注意实验时简述设计思路。

(3) 引起 UDP 丢包的可能原因是什么？

本次实验完成后，请根据组员在实验中的贡献，请实事求是，自评在实验中应得的分数。（按百分制）

(4) (可选) 实验教程实例 3-1.

学号	学生	自评分
22320021	陈安康	100
22318001	蔡可忻	100
22336070	冯乙山	100
22336064	杜雨彤	100

【交实验报告】

上传实验报告：助教

截止日期（不迟于）：1 周之内

上传包括两个文件：

(1) 小组实验报告。上传文件名格式：小组号_编程实验.pdf（由组长负责上传）

例如：文件名“10_编程实验.pdf”表示第 10 组的编程实验报告

(2) 小组成员实验体会。每个同学单独交一份只填写了实验体会的实验报告。只需填写自己的学号和姓名。

文件名格式：小组号_学号_姓名_编程实验.pdf（由组员自行上传）

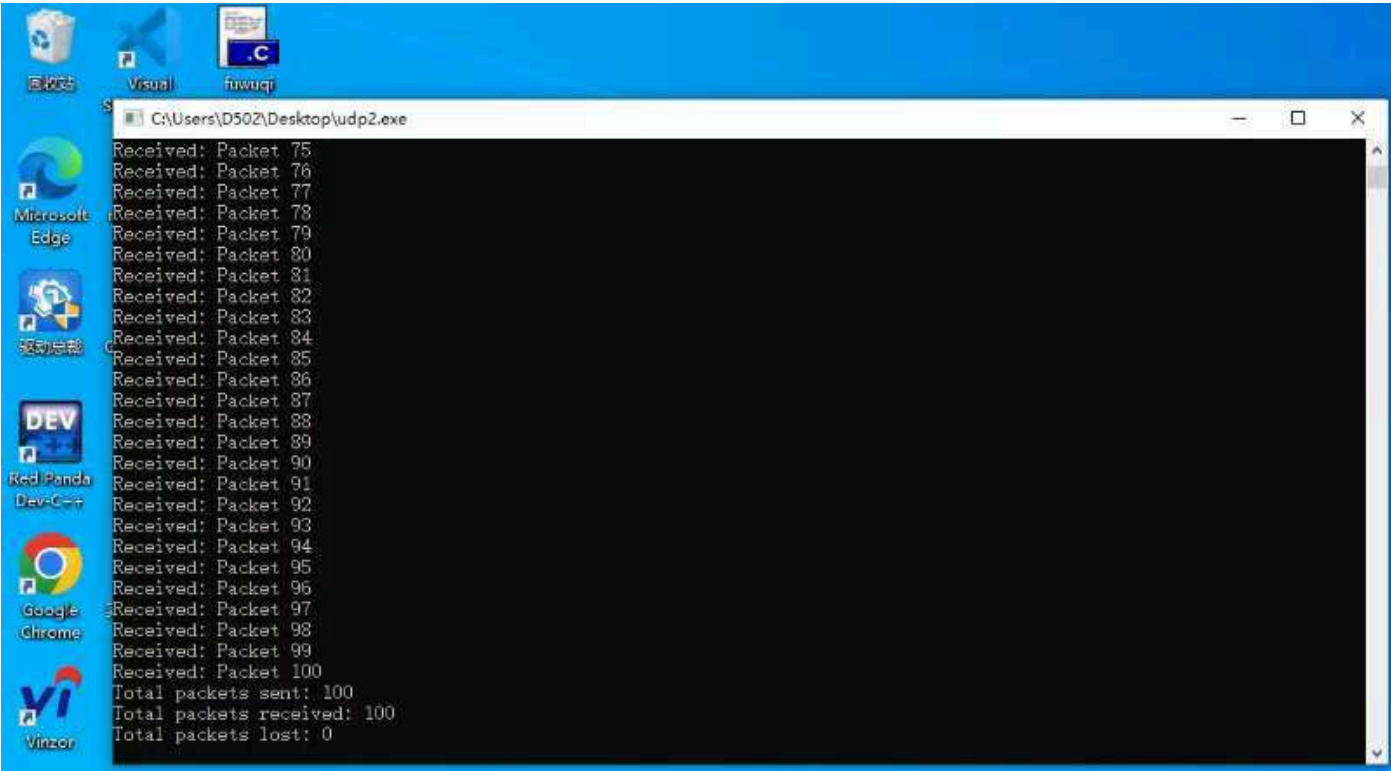
例如：文件名“10_05373092_张三_编程实验.pdf”表示第 10 组的编程实验报告。

注意：不要打包上传！

以下是发送端，由于互联网环境和局域网环境都是在实验室主机上运行的，因此只放了一张图：



以下是局域网环境下接收端：



以下是互联网环境下接收端：

```
C:\Users\86135\Desktop\c++\ X + v
Received: Packet 75
Received: Packet 76
Received: Packet 77
Received: Packet 78
Received: Packet 79
Received: Packet 80
Received: Packet 81
Received: Packet 82
Received: Packet 83
Received: Packet 84
Received: Packet 85
Received: Packet 86
Received: Packet 87
Received: Packet 88
Received: Packet 89
Received: Packet 90
Received: Packet 91
Received: Packet 92
Received: Packet 93
Received: Packet 94
Received: Packet 95
Received: Packet 96
Received: Packet 97
Received: Packet 98
Received: Packet 99
Received: Packet 100
Total packets sent: 100
Total packets received: 100
Total packets lost: 0
```

局域网下发送端的数据报：

*校园网

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

udp

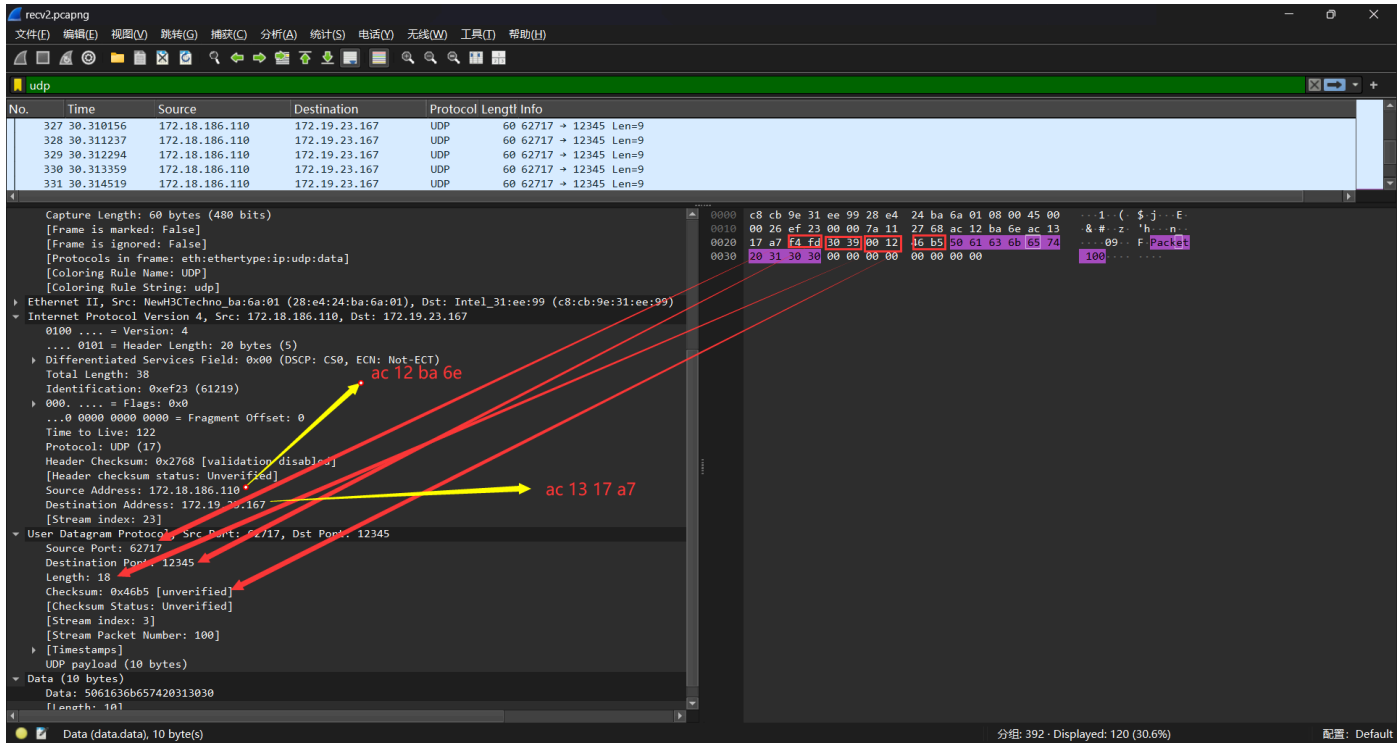
No.	Time	Source	Destination	Protocol	Length	Info
38	-35.372110	172.16.4.2	172.16.4.3	UDP	50	56373 → 12345 Len=8
39	-35.370648	172.16.4.2	172.16.4.3	UDP	50	56373 → 12345 Len=8
40	-35.368834	172.16.4.2	172.16.4.3	UDP	50	56373 → 12345 Len=8
41	-35.366811	172.16.4.2	172.16.4.3	UDP	50	56373 → 12345 Len=8
42	-35.366203	172.16.4.2	172.16.4.3	UDP	50	56373 → 12345 Len=8
43	-35.365584	172.16.4.2	172.16.4.3	UDP	50	56373 → 12345 Len=8
44	-35.364972	172.16.4.2	172.16.4.3	UDP	50	56373 → 12345 Len=8
45	-35.364354	172.16.4.2	172.16.4.3	UDP	50	56373 → 12345 Len=8
46	-35.363740	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
47	-35.363084	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
48	-35.362435	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
49	-35.361774	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
50	-35.361106	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
51	-35.360416	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
52	-35.359728	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
53	-35.359041	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
54	-35.358351	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
55	-35.357663	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
56	-35.356971	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
57	-35.356282	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
58	-35.355592	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9
59	-35.354904	172.16.4.2	172.16.4.3	UDP	51	56373 → 12345 Len=9

No.	Time	Source	Destination	Protocol	Length	Info
99	8.315349	10.8.8.8	172.19.23.167	DNS	160	Standard query response 0xbfe8 A otheve.beacon.qq.com CNAME ins-u4xprfq.ias.tencent-cloud.net A 120.226.166.151 A 120.226.165.14
100	8.315848	172.19.23.167	10.8.8.8	DNS	80	Standard query 0x4d4e AAAA otheve.beacon.qq.com
101	8.325026	10.8.8.8	172.19.23.167	DNS	184	Standard query response 0x4d4e AAAA otheve.beacon.qq.com CNAME ins-u4xprfq.ias.tencent-cloud.net AAAA 2409:8c54:1040:6::20 AAAA
222	23.798960	172.19.23.167	10.8.8.8	DNS	87	Standard query 0x1aec AAAA cn-beijing.log.aliyuncs.com
223	23.801431	10.8.8.8	172.19.23.167	DNS	166	Standard query response 0x1aec AAAA cn-beijing.log.aliyuncs.com SOA hidden-master.aliyun.com
266	30.229254	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=8
267	30.232167	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=8
268	30.233428	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=8
269	30.235852	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=8
270	30.235852	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=8
271	30.237322	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=8
272	30.238465	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=8
273	30.239590	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=8
274	30.240850	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=8
275	30.247128	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
276	30.248212	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
277	30.248212	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
278	30.249300	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
279	30.250397	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
280	30.250397	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
281	30.251765	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
282	30.252566	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
283	30.252566	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
284	30.254219	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
285	30.254219	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
286	30.254678	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
287	30.257056	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
288	30.259151	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
289	30.260412	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
290	30.260412	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
291	30.261530	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
292	30.261530	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
293	30.262589	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
294	30.263660	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
295	30.264757	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
296	30.264757	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
297	30.266852	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
298	30.267972	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
299	30.268989	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
300	30.270081	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9

No.	Time	Source	Destination	Protocol	Length	Info
327	30.310156	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
328	30.311237	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
329	30.312294	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
330	30.313359	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
331	30.314519	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
332	30.315505	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
333	30.316649	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
334	30.318020	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
335	30.319277	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
336	30.320348	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
337	30.321781	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
338	30.322817	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
339	30.325273	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
340	30.327360	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
341	30.328379	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
342	30.329416	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
343	30.330574	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
344	30.331690	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
345	30.332038	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
346	30.335152	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
347	30.336405	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
348	30.337803	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
349	30.337803	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
350	30.340994	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
351	30.342067	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
352	30.343167	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
353	30.344265	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
354	30.344265	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
355	30.346641	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
356	30.346641	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
357	30.348263	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
358	30.348700	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
359	30.351918	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
360	30.352945	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
361	30.354049	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
362	30.355112	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
363	30.356539	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
364	30.358411	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=9
365	30.360551	172.18.186.110	172.19.23.167	UDP	60	62717 → 12345 Len=10

以上截图是互联网环境下udp接收端的接收报文，从266 - 365共接收到100个数据报，接受率100%。

由于是互联网环境，所以源地址不是发送端ip。



以上是最后一个报文的内容，可以看到数据报内容为 “Packet 100” 。

- (1)
- (2)
- (3)
- (4)

(1)

在编程时，编译不通过，报错：

```
cmd /c chcp 65001>nul && "C:\Program Files\x86_64-8.1.0-release-win32-seh-rt_v6-rev0\mingw64\bin\
agnostics-color=always -g C:\Users\86135\Desktop\c++\udp2.c -o C:\Users\86135\Desktop\c++\udp2.e
C:\Users\86135\AppData\Local\Temp\cc5tUyqN.o: In function `main':
C:/Users/86135/Desktop/c++/udp2.c:21: undefined reference to `__imp_WSASStartup'
C:/Users/86135/Desktop/c++/udp2.c:27: undefined reference to `__imp_socket'
C:/Users/86135/Desktop/c++/udp2.c:29: undefined reference to `__imp_WSAGetLastError'
C:/Users/86135/Desktop/c++/udp2.c:30: undefined reference to `__imp_WSACleanup'
C:/Users/86135/Desktop/c++/udp2.c:37: undefined reference to `__imp_htons'
C:/Users/86135/Desktop/c++/udp2.c:40: undefined reference to `__imp_bind'
C:/Users/86135/Desktop/c++/udp2.c:41: undefined reference to `__imp_WSAGetLastError'
C:/Users/86135/Desktop/c++/udp2.c:42: undefined reference to `__imp_closesocket'
C:/Users/86135/Desktop/c++/udp2.c:43: undefined reference to `__imp_WSACleanup'
```

随后通过查阅资料发现，这是因为链接器无法找到Winsock库中的必要函数，因此在编译命令后面加入-lws2_32，告诉链接器链接到名为ws2_32的库，解决问题：

```
PS C:\Users\86135\Desktop\c++> g++ udp.c -o udp.exe -lws2_32
PS C:\Users\86135\Desktop\c++>
```

在最初编程时，接收端总是接收不到消息，在查找相关资料后发现，是在将ip以及端口号写入sockaddr_in数据结构时没能注意到网络传输时的存储方式（大端存储）与主机存储方式（一般是小端存储）的不同，导致一直发送不到正确的主机上，造成数据接收失败。

随后我们使用htons和inet_addr函数来成功解决这一问题。

(2)

udp接收端代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
```

```

#define PORT 12345
#define PACKET_SIZE 100
#define BUFFER_SIZE 512
#define PACKET_COUNT 100

int main() {
    WSADATA wsaData;
    SOCKET recvSocket;
    struct sockaddr_in localAddr, clientAddr;
    int clientAddrLen = sizeof(clientAddr);
    char buffer[BUFFER_SIZE];
    int receivedPackets = 0;

    // 初始化Winsock库
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        fprintf(stderr, "WSAStartup failed\n");
        return 1;
    }

    recvSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (recvSocket == INVALID_SOCKET) {
        fprintf(stderr, "socket failed: %ld\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }

    memset(&localAddr, 0, sizeof(localAddr));
    localAddr.sin_family = AF_INET;
    localAddr.sin_port = htons(PORT);
    localAddr.sin_addr.s_addr = inet_addr("172.16.4.3"));

    if (bind(recvSocket, (struct sockaddr*)&localAddr, sizeof(localAddr)) ==
    SOCKET_ERROR) {
        fprintf(stderr, "bind failed: %ld\n", WSAGetLastError());
        closesocket(recvSocket);
        WSACleanup();
        return 1;
    }

    int c = 0;
    while (c < PACKET_COUNT) {
        int bytesReceived = recvfrom(recvSocket, buffer, BUFFER_SIZE - 1, 0,
        (struct sockaddr*)&clientAddr, &clientAddrLen);
        if (bytesReceived > 0) {
            buffer[bytesReceived] = '\0';

            if (clientAddr.sin_addr.s_addr == inet_addr("172.26.46.52")) {
                printf("Received: %s\n", buffer);
                receivedPackets++;
            } else {
                printf("not expect packet\n");
            }
        }
    }
}

```

```
        } else {
            fprintf(stderr, "recvfrom failed: %ld\n", WSAGetLastError());
            break;
        }
        c++;
    }

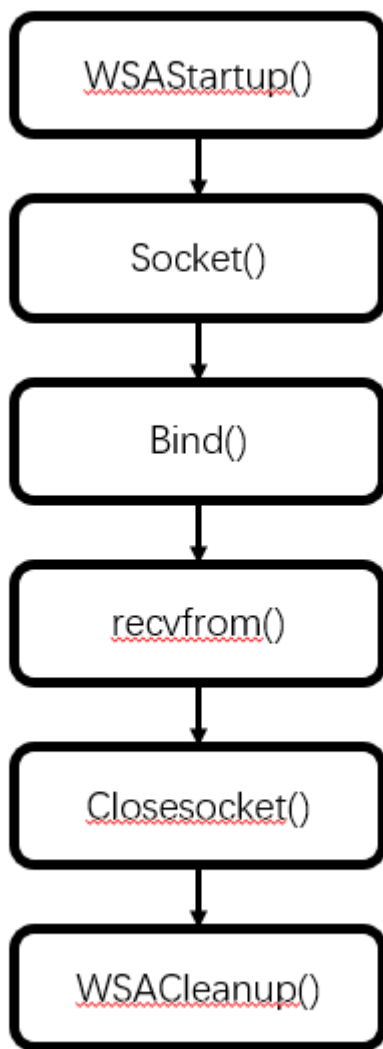
    int lostPackets = PACKET_COUNT - receivedPackets;

    printf("Total packets sent: %d\n", PACKET_COUNT);
    printf("Total packets received: %d\n", receivedPackets);
    printf("Total packets lost: %d\n", lostPackets);

    closesocket(recvSocket);
    WSACleanup();
    while(1) {

    }
    return 0;
}
```

接收端（服务端）程序详细流程图如下：



首先使用WSAStartup(MAKEWORD(2, 2), &wsaData)初始化WSA库，接着使用socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)创建UDP套接字，使用Bind()绑定本地ip。

设置循环，该循环只有在收到一百个数据报后才会退出（不管收到的数据报是不是发送端发的，接收到100个就退出循环）。在每次循环时，使用recvfrom()函数接收数据报，此函数会将接收到的数据报放入缓冲中buffer（每次都是从初始位置放，意味着每次接收新的数据报后，buffer就会被覆盖，不存储之前的内容），并且将发送端的ip地址和端口号等信息写入chlientAddr中。

读取chlientAddr中的地址，判断是不是预想中的发送方地址（这里理想的认为发送方ip不会再有其他端口与该套接字通过udp通信），若是，计数器加1。

100次循环结束后，统计丢包率等信息，关闭套接字，释放所使用的Windows Sockets DLL。

发送端代码设计如下：

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>

#define PORT 12345
#define SERVER_IP "172.16.4.3" // 接收端的IP地址
#define PACKET_COUNT 100
#define PACKET_SIZE 100

int main() {
    WSADATA wsaData;
    SOCKET sendSocket;
    struct sockaddr_in serverAddr;
    char packet[PACKET_SIZE];
    int i;

    // 初始化Winsock库
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        fprintf(stderr, "WSAStartup failed\n");
        return 1;
    }

    // 创建发送套接字
    sendSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sendSocket == INVALID_SOCKET) {
        fprintf(stderr, "socket failed: %ld\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }

    // 设置接收端地址和端口
    memset(&serverAddr, 0, sizeof(serverAddr));
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr(SERVER_IP);

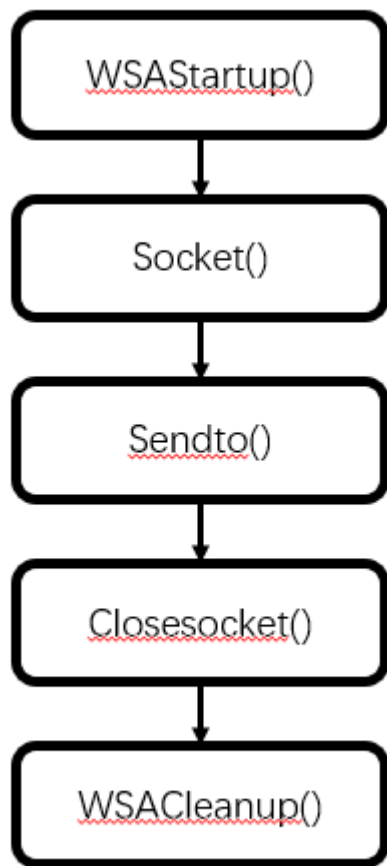
    // 发送数据包
    for (i = 0; i < PACKET_COUNT; i++) {
        sprintf(packet, "Packet %d", i + 1);
        if (sendto(sendSocket, packet, strlen(packet), 0, (struct
sockaddr*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
            fprintf(stderr, "sendto failed: %ld\n", WSAGetLastError());
            closesocket(sendSocket);
            WSACleanup();
            return 1;
        }
        printf("Sent: %s\n", packet);
    }

    // 关闭套接字和清理Winsock库
    closesocket(sendSocket);
    WSACleanup();
    while(1) {

}

```

```
    return 0;  
}
```



发送端代码十分简单，首先使用WSAStartup(MAKEWORD(2, 2), &wsaData)初始化WSA库，接着使用socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)创建UDP套接字，设置好接收端的ip和端口号，在100次循环中使用sendto()发送指定的数据报即可。最后关闭套接字，释放所使用的Windows Sockets DLL。

(3)

使用Socket API时，客户端与服务端都有可能共同使用的函数，如Send()和Recv(),Socket(), WSAStartup(), WSACleanup()等。

但它们还有一些函数是不同的，如服务端使用的bind()函数，用来将本地ip和端口绑定到套接字上，若为TCP连接，则还需要listen()函数请求TCP进程监听套接字设置的端口，建立连接请求队列。accept()函数来获得客户端请求。

对于客户端，若是TCP连接，客户端需要通过connect函数发出连接请求。

(4)

struct sockaddr * addr 用来存储ip和端口，其中包含两个部分，sa_family用于保存地址族，用于指定地址类型，如AF_INET (IPv4) 、 AF_INET6 (IPv6) 等。

sa_data 用于存储与套接字相关的地址数据，包含地址和端口信息。一般我们使用struct sockaddr_in来存储，在上传参数时通过类型转换将sockaddr_in *转换成struct sockaddr *类型进行参数传递。

struct sockaddr 中的sa_data是一个14字节的数据结构，则一共可用112位，前16位用来表示端口号，中间32位用来表示IP地址，后64位不使用。比如，若sa_data如下：

0x0050ac100403(网络传输中采用大端存储，而一般主机采用小端存储)前16位表示端口号，因为是大端存储，所以端口号是80，后面的0xac100403表示ip地址，是172.16.4.3。

- (5)
 - 面向连接的Socket通常使用TCP。
 - 面向非连接的Socket通常使用UDP。
- (6)
 - 说明面向连接的客户端和面向非连接的客户端在收发数据时有什么区别
 - 面向非连接的客户端如何判断数据发送结束
- 创建一个 TCP Socket
- 阻塞等待客户端连接
- 阻塞接收数据
- 创建一个 TCP Socket
- 设置为非阻塞模式
- 使用 select 进行非阻塞监听

(5)

面向连接的Socket通常使用TCP。

在建立连接的过程中，客户端与服务器之间需要进行“三次握手”的过程，只有在连接成功建立之后，才能开始数据传输，当数据传输完毕，经过“四次挥手”结束连接。

在建立时：客户端首先创建一个Socket对象，调用connect（）方法，向服务器发送连接请求。

1 0.000000	172.16.39.73	172.16.28.58	TCP	62 1372 → 21 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM
2 0.000340	172.16.28.58	172.16.39.73	TCP	62 21 → 1372 [SYN, ACK] Seq=0 Ack=1 Win=16384 Len=0 MSS=1460 SACK_PERM
3 0.000385	172.16.39.73	172.16.28.58	TCP	54 1372 → 21 [ACK] Seq=1 Ack=1 Win=65535 Len=0

- 客户端发送一个SYN包到服务器，表示请求建立连接。
- 服务器收到SYN包后，回复一个SYN-ACK包，表示同意建立连接。
- 客户端收到SYN-ACK包后，发送一个ACK包，连接建立完成。

面向非连接的Socket通常使用UDP。

UDP在发送数据之前不需要建立连接，每个数据包都独立，发送方与接收方之间没有固定的连接，在传输的过程中发送端为每一个数据包分配一个ID，当接收端收到数据包时，再给发送端返回数据包，告诉发送端已接收。

在建立时:客户端创建一个Socket对象,再使用`sendto()`方法发送数据到服务器,而不需要先建立连接。

代码如下:

```
// 创建发送套接字
sendSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
if (sendSocket == INVALID_SOCKET) {
    fprintf(stderr, "socket failed: %ld\n", WSAGetLastError());
    WSACleanup();
    return 1;
}

.....

// 发送数据包
for (i = 0; i < PACKET_COUNT; i++) {
    sprintf(packet, "Packet %d", i + 1);
    if (sendto(sendSocket, packet, strlen(packet), 0, (struct
sockaddr*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
        fprintf(stderr, "sendto failed: %ld\n", WSAGetLastError());
        closesocket(sendSocket);
        WSACleanup();
        return 1;
    }
    printf("Sent: %s\n", packet);
}
```

(6)

说明面向连接的客户端和面向非连接的客户端在收发数据时有什么区别

面向连接的Socket使用TCP协议,提供可靠的、面向字节流的通信。数据在发送和接收时,TCP会确保以下几点:

1. 可靠性: TCP会对发送的数据进行确认 (ACK), 如果发送的数据包丢失, TCP会自动重传。
2. 顺序性: TCP会确保数据按照发送的顺序到达接收方。
3. 流量控制: TCP会根据网络的拥塞情况调整数据的发送速率, 避免网络拥堵。

数据收发过程:

客户端通过 `send()` 方法发送数据，TCP会将数据分段并进行封装。服务器通过 `recv()` 方法接收数据，TCP会将接收到的数据重新组装成完整的字节流。

面向非连接的Socket使用UDP协议，提供不可靠的、面向数据报的通信。UDP的特点包括：

1. 不可靠性：UDP不保证数据包的送达，也不进行重传。
2. 无序性：UDP不保证数据包的顺序，接收方可能会以不同的顺序接收到数据。
3. 无连接：UDP在发送数据之前不需要建立连接。

数据收发过程：

客户端通过 `sendto()` 方法发送数据，UDP会将数据打包并直接发送到指定的地址。服务器通过 `recvfrom()` 方法接收数据，UDP会将接收到的数据和发送方的地址一起返回。

代码：

```
// 创建发送套接字
sendSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
if (sendSocket == INVALID_SOCKET) {
    fprintf(stderr, "socket failed: %ld\n", WSAGetLastError());
    WSACleanup();
    return 1;
}

.....

// 发送数据包
for (i = 0; i < PACKET_COUNT; i++) {
    sprintf(packet, "Packet %d", i + 1);
    if (sendto(sendSocket, packet, strlen(packet), 0, (struct
sockaddr*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
        fprintf(stderr, "sendto failed: %ld\n", WSAGetLastError());
        closesocket(sendSocket);
        WSACleanup();
        return 1;
    }
    printf("Sent: %s\n", packet);
}

.....

//接受数据包
int c = 0;
while (c < PACKET_COUNT) {
    int bytesReceived = recvfrom(recvSocket, buffer, BUFFER_SIZE - 1, 0,
(struct sockaddr*)&clientAddr, &clientAddrLen);
    if (bytesReceived > 0) {
        buffer[bytesReceived] = '\0';
```



```
        if (clientAddr.sin_addr.s_addr == inet_addr("172.26.46.52")) {
            printf("Received: %s\n", buffer);
            receivedPackets++;
        } else {
            printf("not expect packet\n");
        }
    } else {
        fprintf(stderr, "recvfrom failed: %ld\n", WSAGetLastError());
        break;
    }
    c++;
}
```

面向非连接的客户端如何判断数据发送结束

在UDP中，数据的发送结束通常是由应用层协议来定义的。UDP本身并没有提供数据结束的机制，因此需要在应用层进行约定。常见的做法包括：

1. 固定长度：发送的数据包长度是固定的，接收方根据预先约定的长度来判断数据的结束。
2. 特殊标志：在数据中添加一个特殊的结束标志（例如特定的字符或字节序列），接收方在接收到这个标志时判断数据的结束。
3. 超时机制：接收方可以设置一个超时时间，如果在一定时间内没有接收到新的数据包，则认为数据发送结束。

(7) 面向连接的通信 定义：面向连接的通信（如 TCP）在数据传输之前需要建立一个连接，确保双方可以进行可靠的数据交换。

优点： 可靠性：面向连接的通信提供数据传输的可靠性，确保数据包按顺序到达，并且没有丢失。 流量控制：可以根据接收方的处理能力进行流量控制，避免数据过载。 拥塞控制：能够检测网络拥塞并采取措施调整数据传输速率。 错误检测与重传：提供错误检测机制，确保数据的完整性，丢失的数据包会被重传。 **缺点：** 延迟：建立连接和维护连接的过程会增加延迟，尤其是在高延迟网络中。 资源消耗：需要维护连接状态，消耗更多的系统资源（如内存和处理器）。 复杂性：实现和管理连接的复杂性较高。 **适用场合：** 适用于需要高可靠性和数据完整性的应用，如文件传输（FTP）、电子邮件（SMTP）和网页浏览（HTTP）。 **无连接通信 定义：**无连接通信（如 UDP）在数据传输时不需要建立连接，数据包独立发送，不保证到达顺序和完整性。

优点： 低延迟：由于不需要建立连接，数据可以快速发送，适合实时应用。 资源消耗少：不需要维护连接状态，节省系统资源。 简单性：实现相对简单，适合快速开发和部署。 **缺点：** 不可靠性：数据包可能丢失、重复或乱序，无法保证数据的完整性。 缺乏

流量控制和拥塞控制：无法根据网络状况调整数据传输速率，可能导致网络拥塞。 错误检测有限：虽然可以实现简单的错误检测，但没有重传机制。 适用场合：适用于对实时性要求高、对可靠性要求低的应用，如视频会议、在线游戏、实时语音通话和流媒体传输。 总结 面向连接的通信和无连接通信各有优缺点，选择合适的通信方式取决于具体的应用需求。对于需要高可靠性和数据完整性的场合，面向连接的通信是更好的选择；而对于实时性要求高、对数据丢失不敏感的应用，无连接通信则更为合适。

(8)

1. 阻塞方式（Blocking Mode） 定义：在阻塞模式下，当调用 Socket 的某些方法（如 `accept()`、`recv()`、`send()` 等）时，如果没有数据可用，程序会等待（阻塞）直到有数据可用或连接建立。

特点：

简单易用：编程模型简单，代码逻辑清晰，适合初学者。 等待时间：在没有数据到达时，调用会一直等待，可能导致程序响应变慢。 适合场景：适用于对实时性要求不高的应用，如简单的客户端-服务器模型。 示例：

```
import socket
```

创建一个 TCP Socket

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('localhost', 8080)) server_socket.listen(1)
```

阻塞等待客户端连接

```
client_socket, addr = server_socket.accept() print(f"Connection from {addr}")
```

阻塞接收数据

```
data = client_socket.recv(1024) print(f"Received: {data.decode()}")
```

2. 非阻塞方式（Non-blocking Mode） 定义：在非阻塞模式下，当调用 Socket 的方法时，如果没有数据可用，程序不会等待，而是立即返回。可以通过轮询或其他机制来检查数据是否可用。

特点：

提高响应性：程序不会因为等待而阻塞，可以同时处理其他任务，提高了应用的响应性。复杂性：编程模型相对复杂，需要处理数据可用性检查和可能的异常。适合场景：适用于对实时性要求高的应用，如聊天应用、在线游戏和实时数据处理。示例：

```
import socket
import select
```

创建一个 TCP Socket

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('localhost', 8080))
server_socket.listen(1)
```

设置为非阻塞模式

```
server_socket.setblocking(False)
```

使用 select 进行非阻塞监听

```
while True:
    try:
        client_socket, addr = server_socket.accept()
        print(f"Connection from {addr}")
    except BlockingIOError: # 没有连接请求，继续循环
        pass
```

```
# 检查是否有数据可读
readable, _, _ = select.select([client_socket], [], [], 0)
for s in readable:
    data = s.recv(1024)
    if data:
        print(f"Received: {data.decode()}")
```

3. 阻塞与非阻塞的比较
- | 阻塞 | 非阻塞 | 特性 | 阻塞方式 | 非阻塞方式 | 等待行为 | 调用会等待 |
|---------|------------|------------------|-----------|-------|--------------|---------------|
| 直到有数据可用 | 调用立即返回 | 不等待 | 编程复杂性简单 | 易于理解 | 复杂，需要处理状态和异常 | 响应性可能导致程序响应变慢 |
| 提高响应性 | 可以同时处理其他任务 | 适用场景简单的客户端-服务器应用 | 实时性要求高的应用 | | | |

引起UDP丢包的可能原因主要包括以下几点：

1. 网络拥塞：当网络流量过大时，路由器和交换机可能无法处理所有传入的数据包，导致一些数据包被丢弃。这种情况在高流量的网络环境中尤为常见。
2. MTU（最大传输单元）限制：UDP数据包的大小受到MTU的限制。如果发送的数据包超过了网络的MTU，数据包可能会被分割或丢弃。通常，UDP数据包的大小应控

制在1500字节以内，以避免分片。

3. 接收缓冲区溢出：每个UDP Socket都有一个接收缓冲区。如果接收方处理数据的速度慢于发送方发送数据的速度，接收缓冲区可能会被填满，导致新的数据包被丢弃。可以通过调整Socket的接收缓冲区大小来缓解这个问题。
4. 网络设备故障：网络中的硬件故障（如路由器、交换机或网卡故障）可能导致数据包丢失。这种情况通常需要检查网络设备的状态和性能。
5. 不可靠的传输：UDP本身不提供重传机制，因此如果数据包在传输过程中丢失，发送方不会知道，也不会尝试重发。这使得UDP在某些情况下更容易出现丢包现象。
6. 防火墙和安全设置：某些防火墙或安全设备可能会阻止UDP数据包，尤其是在特定端口或协议上。这可能导致数据包在到达目的地之前被丢弃。
7. 发送方和接收方的处理速度不匹配：如果发送方发送数据的速度远高于接收方处理数据的速度，可能会导致数据包的丢失。接收方需要有足够的能力来处理接收到的数据。
8. 物理层问题：网络连接的物理层问题（如电缆损坏、信号干扰等）也可能导致数据包丢失。这种情况通常需要检查物理连接和网络环境。